TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

CẨM NANG LUYỆN NÓI & PHẢN BIỆN CÁ NHÂN
VÕ NGỌC MINH ĐẠT (1923040088)

Vai trò: Desktop Software Engineer – Phụ trách Windows Forms, Sự cố & Kết luận | Thứ tự thuyết trình: Người số 4 (Trình bày WinForms, Xử lý Sự cố & KẾT THÚC) | Thời lượng mục tiêu: 4 phút 45 giây (Khóa cứng thời gian)
Dự án: Hệ thống Quản lý Hoạt động Hội nhập – VISIONARY

📌 **CHIẾN THUẬT DÀNH CHO MINH ĐẠT:** Đạt là người chốt hạ toàn bộ bài thuyết trình 15 phút. Nội dung phụ trách mang tính chuyên sâu và quyết định điểm số cao: (1) Windows Forms C# & async/await; (2) Demo điểm danh trực tiếp kết nối với dữ liệu Android; (3) 3 khác biệt kỹ thuật giữa 3 nền tảng; (4) Phân tích sự cố thật NETSDK1100 trên GitHub Actions; (5) Nêu 3 hạn chế thực tế, hướng phát triển 2 tuần và LỜI CẢM ƠN KẾT THÚC. Sau lời cảm ơn của Đạt là dừng lại, không chuyển lại cho Đức!

I. PHÂN BỔ SLIDE VÀ THỜI GIAN (4 PHÚT 45 GIÂY)

Slide	Tiêu đề Slide	Nội dung trọng tâm	Thời lượng
Slide 12	Ứng dụng Windows Forms C#	Kiến trúc WinForms .NET 8, HttpClient tái sử dụng, cơ chế async/await, bảng điều khiển	55 giây
Slide 13	Demo WinForms Quản Trị Trực Tiếp	Mở app biên dịch sẵn, đăng nhập BCN, nạp danh sách đăng ký sự kiện & bấm điểm danh	1 phút 05 giây
Slide 14	3 Điểm Khác Biệt Giữa 3 Nền Tảng	So sánh đối chiếu: Thư viện mạng, Điều phối UI thread, Lưu trữ token; Điểm buộc phải giống	55 giây
Slide 15	Một Sự Cố Thật & Cách Sửa	Phân tích lỗi NETSDK1100 trên GitHub Actions Ubuntu CI: Log thật -> Nguyên nhân -> Giải pháp	1 phút 00 giây
Slide 16	Giới Hạn, Hướng Phát Triển & KẾT THÚC	3 hạn chế thực tế, kế hoạch 2 tuần, lời cảm ơn Hội đồng & kết thúc bài nói (KHÔNG chuyển lại)	50 giây

II. LỜI THUYẾT TRÌNH TỪNG CÂU (HỌC THUỘC LÒNG & NÓI TỰ NHIÊN)

1. Khi ở Slide 12 (Windows Forms C#) – 55 giây:

"Kính chào Thầy và các bạn, em là Võ Ngọc Minh Đạt. Tiếp nối phần phân tích API của Hồ Khang, em xin giới thiệu ứng dụng Windows Forms dành cho Ban Chủ nhiệm quản trị trên máy tính.

Ứng dụng được xây dựng trên nền tảng .NET 8 C#. Toàn bộ các thao tác mạng được xử lý qua lớp ApiService dùng chung một đối tượng HttpClient duy nhất theo mẫu Singleton, giúp tránh hoàn toàn hiện tượng cạn kiệt cổng kết nối (Socket Exhaustion).

Đặc biệt, WinForms ứng dụng mô hình bất đồng bộ async/await nguyên bản của C#. Từ khóa await tự động giải phóng giao diện trong lúc chờ phản hồi từ server và tự động điều phối luồng quay về đúng UI Thread khi có dữ liệu. Nhóm tuyệt đối không sử dụng .Result hay .Wait() vì sẽ gây Deadlock khóa cứng giao diện."

2. Khi ở Slide 13 (Demo WinForms Quản Trị Trực Tiếp) – 1 phút 05 giây:

"Sau đây em xin demo trực tiếp ứng dụng WinForms đã được biên dịch sẵn.

[ĐẠT THAO TÁC LIVE TRÊN WINFORMS]:
- Bước 1: Mở ứng dụng Visionary.Desktop.exe. Đăng nhập với tài khoản Ban Chủ nhiệm.
- Bước 2: Tại danh sách sự kiện, em chọn 'Hội thảo Kỹ năng Hội nhập Quốc tế' và bấm nút 'Xem danh sách đăng ký' (btnXemDangKy).
- Bước 3: Trên DataGridView (grdDangKy), Thầy có thể thấy ngay bản ghi đăng ký mà bạn Trí Khang vừa thực hiện trên Android ở phần trước đã xuất hiện tức thì.
- Bước 4: Em bấm nút 'Xác nhận Điểm danh' (btnDiemDanh). WinForms gửi request POST /api/events/{id}/attendance. Giao diện cập nhật trạng thái 'Đã tham gia' với tick xanh thành công.

Điều này chứng minh ba ứng dụng hoạt động hoàn toàn đồng bộ trên cùng một nguồn dữ liệu duy nhất."

3. Khi ở Slide 14 (3 Điểm Khác Biệt Giữa 3 Nền Tảng) – 55 giây:

"Thưa Thầy, dù cùng kết nối vào một Web API, ba nền tảng có 3 điểm khác biệt kỹ thuật mang tính bắt buộc:
1. Thư viện mạng & Bất đồng bộ: Web dùng Fetch API với Promise; WinForms dùng HttpClient với cú pháp async/await của C#; còn Android dùng OkHttp/Retrofit với mô hình hàng đợi Callback.
2. Điều phối luồng giao diện: C# async/await tự đồng bộ SynchronizationContext về UI Thread; trong khi Android bắt buộc lập trình viên phải tự chuyển đổi qua Handler(Looper.getMainLooper()).
3. Môi trường & Lưu trữ Token: Web lưu trên LocalStorage của trình duyệt; WinForms lưu trong biến static của tiến trình Desktop; còn Android lưu vào SharedPreferences bền vững của thiết bị.

Ngược lại, những thứ BẮT BUỘC GIỐNG NHAU TUYỆT ĐỐI là: Cấu trúc JSON, đường dẫn Endpoint, phương thức HTTP, mã trạng thái và quy tắc nghiệp vụ kiểm tra quyền trên server."

4. Khi ở Slide 15 (Một Sự Cố Thật & Cách Sửa) – 1 phút 00 giây:

"Tiếp theo, em xin báo cáo về một sự cố kỹ thuật có thật mà nhóm đã gặp phải trong quá trình thiết lập CI/CD.

Khi cấu hình GitHub Actions chạy tự động trên môi trường Linux Ubuntu, quy trình build Backend bị gãy

với thông điệp lỗi nguyên văn: 'NETSDK1100: To build a project targeting Windows on this operating system, set the EnableWindowsTargeting property to true.'

Nguyên nhân gốc rễ: File giải pháp Visionary.sln ban đầu chứa cả project WinForms (nhắm mục tiêu net8.0-windows). Khi chạy trên Ubuntu, .NET SDK Linux không hỗ trợ build thành phần Windows Desktop.

Quy trình khắc phục của nhóm: Nhóm tách riêng một file giải pháp backend độc lập mang tên Visionary.Backend.sln chỉ chứa API, Core và Infrastructure. Pipeline GitHub Actions được cấu hình trỏ vào file sln này. Kết quả kiểm chứng: Pipeline Backend CI trên Ubuntu vượt qua 100% tất cả 15 bài kiểm thử đơn vị và tích hợp, trong khi WinForms được đóng gói trên máy Windows riêng biệt."

5. Khi ở Slide 16 (Giới Hạn, Hướng Phát Triển & KẾT THÚC) – 50 giây:

"Nhìn nhận một cách thẳng thắn và khách quan, hệ thống của nhóm vẫn còn 3 hạn chế thực tế:
1. Chưa có bộ kiểm thử tự động tích hợp xuyên suốt đồng thời cả 3 nền tảng (E2E End-to-End).
2. Chưa hỗ trợ Push Notification thời gian thực qua SignalR hay Firebase khi có sự kiện khẩn cấp.
3. Chưa có bộ nhớ đệm Offline Cache trên thiết bị di động khi mất kết nối mạng.

Nếu có thêm 2 tuần, nhóm ưu tiên tích hợp SignalR để đồng bộ thông báo điểm danh tức thì và bổ sung bộ test tự động Playwright cho Web và Espresso cho Android.

Đến đây, nhóm VISIONARY API xin chân thành cảm ơn Quý Thầy Cô trong Hội đồng và các bạn đã chú ý lắng nghe. Chúng em rất mong nhận được những câu hỏi và đóng góp quý báu từ Thầy Cô. Chúng em xin trân trọng cảm ơn! [KẾT THÚC TẠI ĐÂY - KHÔNG CHUYỂN LẠI CHO ĐỨC]."

III. MÃ NGUỒN CẦN MỞ VÀ 3 ĐIỂM NÓNG GIẢNG VIÊN DỄ TRỎ VÀO

Tệp mã nguồn Đạt phụ trách mở khi giảng viên kiểm tra WinForms & CI/CD:

📁 File WinForms FormMain: desktop/Visionary.Desktop/Forms/FormMain.cs
📁 File API Service WinForms: desktop/Visionary.Desktop/Services/ApiService.cs
📁 File Workflow CI: .github/workflows/backend.yml

📘 Mã nguồn minh chứng: Cơ chế HttpClient Singleton và async/await trong ApiService.cs

```
// desktop/Visionary.Desktop/Services/ApiService.cs
public class ApiService {
    // Tái sử dụng HttpClient duy nhất để tránh cạn kiệt socket
    private static readonly HttpClient _httpClient = new HttpClient();

    public async Task<ApiResponse<T>> GetAsync<T>(string endpoint) {
        var request = new HttpRequestMessage(HttpMethod.Get, _baseUrl + endpoint);
        if (!string.IsNullOrEmpty(_token))
            request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", _token);

        var response = await _httpClient.SendAsync(request);
        var json = await response.Content.ReadAsStringAsync();
        return JsonSerializer.Deserialize<ApiResponse<T>>(json, _jsonOptions);
    }
```

```
}
```

👉 Điểm 1: Dòng 'private static readonly HttpClient _httpClient = new HttpClient();':

Giảng viên hỏi: Tại sao lại dùng biến static? Tại sao không dùng 'using var client = new HttpClient()' mỗi lần gọi?
-> Đạt trả lời: 'Dạ thưa Thầy, HttpClient được thiết kế để sống lâu trong suốt vòng đời ứng dụng. Mặc dù triển khai IDisposable, nhưng khi gọi Dispose, socket bên dưới hệ điều hành không đóng ngay mà rơi vào trạng thái TIME_WAIT trong 240 giây. Nếu khởi tạo liên tục trong 'using', ứng dụng sẽ nhanh chóng cạn kiệt cổng socket (Socket Exhaustion) và ném SocketException.'

👉 Điểm 2: Câu hỏi vì sao không dùng .Result hoặc .Wait():

Giảng viên hỏi: Nếu thay 'await _httpClient.SendAsync()' bằng '.Result' thì chuyện gì xảy ra?
-> Đạt trả lời: 'Dạ thưa Thầy, việc gọi .Result trên WinForms sẽ chặn (block) Main UI Thread để chờ Task hoàn thành. Tuy nhiên, Task đó khi chạy xong lại cần SynchronizationContext của UI Thread để tiếp tục. Nhưng UI Thread lúc này đang bị .Result chiếm giữ để chờ, kết quả là hai bên chờ lẫn nhau tạo thành DEADLOCK, cửa sổ ứng dụng bị treo đơ vĩnh viễn không có thông báo lỗi nào.'

👉 Điểm 3: Khắc phục sự cố GitHub Actions Backend CI Run #12 (13/15 Passed -> 15/15 Passed Run #40231783994):

Giảng viên hỏi: Nhóm đã phân tích và sửa hai bài test thất bại trên GitHub Actions Run #12 như thế nào?
-> Đạt trả lời: 'Dạ thưa Thầy, trong Run #12, test Search (dòng 168) và test GetBySlug (dòng 183) bị fail do chu trình SeedAsync bị hủy ngang khi gặp lỗi SQLite Error 19 UNIQUE constraint failed: Roles.Name. Nguyên nhân do hai test class chạy song song cùng tranh chấp file visionary.db. Nhóm em đã giải quyết triệt để bằng cách thêm SemaphoreSlim(1, 1) trong DbInitializer.SeedAsync, tạo CustomWebApplicationFactory cấp phát database SQLite biệt lập cho mỗi lần test và gộp chung [Collection(IntegrationTests)]. Sau khi commit c7af080, GitHub Actions Run #40231783994 đã vượt qua 100% cả 15/15 bài test trên runner Linux x64.'

IV. BỘ CÂU HỎI PHẢN BIỆN CHUYÊN SÂU WINFORMS & TỔNG QUAN

❓ Câu 1: Vì sao tên sự kiện trong WinForms là btnDiemDanh_Click thay vì đổi thành diemDanh cho giống Android và Web?

🗣️ Đạt trả lời (20-30s): Dạ thưa Thầy, trong Windows Forms, Visual Studio tự động sinh tên hàm sự kiện theo định dạng <TênControl>_<SựKiện> và gắn chặt với tệp Designer.cs. Nếu cố tình đổi tên hàm thì sẽ mất liên kết với Designer. Do đó theo quy ước của môn học, WinForms chấp nhận giữ nguyên tên sự kiện, còn các hàm nghiệp vụ bên trong thì tuân thủ bộ động từ chung.

❓ Câu 2: Nếu mạng bị chập chờn hoặc timeout thì WinForms xử lý thế nào?

🗣️ Đạt trả lời (20-30s): Dạ thưa Thầy, HttpClient được cấu hình Timeout = TimeSpan.FromSeconds(15). Nếu quá thời gian hoặc mất kết nối, HttpClient sẽ ném HttpRequestException hoặc TaskCanceledException. Lớp ApiService bắt ngoại lệ này, gán MaTrangThai = 0 và trả về đối tượng ApiResponse với thông báo tiếng Việt rõ ràng thay vì làm crash ứng dụng.

❓ Câu 3: Bảng điều khiển DataGridView trên WinForms có ưu điểm gì so với Web và Android?

🗣️ Đạt trả lời (20-30s): Dạ thưa Thầy, DataGridView trên WinForms hỗ trợ hiển thị bảng dạng bảng tính nhiều cột với hiệu năng cao, tự động sắp xếp (Sort), căn chỉnh độ rộng cột và tích hợp sẵn DataBinding với BindingSource, rất phù hợp cho nghiệp vụ quản trị nhiều dòng dữ liệu của Ban Chủ nhiệm.

❓ Câu 4: Chỉ một dòng do AI sinh ra mà nhóm đã sửa. Sửa gì và vì sao phải sửa?

🗣️ Đạt trả lời (20-30s): Dạ thưa Thầy, AI ban đầu đã sinh mã Retrofit trên Android với kiểu trả về là ApiResponse<List<LearningResourceItem>>. Nhưng thực tế Backend ASP.NET Core trả về cấu trúc phân trang ApiResponse<PagedResponse<LearningResourceItem>>. Kết quả là ứng dụng Android bị lỗi ánh xạ JSON và không hiển

thị được danh sách. Nhóm đã phát hiện qua log, sửa lại Generic Type thành PagedResponse và đọc data.getItems(), ứng dụng đã hoạt động trơn tru.

V. CHECKLIST TỰ LUYỆN TRƯỚC BUỔI BẢO VỆ

- [ ] Mở sẵn ứng dụng Visionary.Desktop.exe đã biên dịch từ desktop/Visionary.Desktop/bin/Debug/net8.0-windows/.
- [ ] Mở sẵn VS Code hoặc Visual Studio trỏ đúng vào FormMain.cs, ApiService.cs và .github/workflows/backend.yml.
- [ ] Tập nói lưu loát từ Slide 12 đến 16, bấm giờ đúng dưới 4 phút 45 giây.
- [ ] Nắm chắc log lỗi NETSDK1100 và giải pháp chia solution backend.
- [ ] Khắc ghi trong đầu: Sau lời cảm ơn là KẾT THÚC bài thuyết trình, KHÔNG chuyển lại cho Đức!